JAVA PROGRAMMING

Core Concepts

Variables · Data Types · Operators · Input/Output · Type Casting

Variables | Data Types | Operators | Input / Output | Type Casting

A clear, example-driven guide to Java fundamentals for beginners and students.

Table of Contents

Chapter 1

Variables

Storing and managing data in Java

1.1 What is a Variable?

A **variable** is a named container in memory that holds a value. Every variable in Java has a **type**, a **name**, and a **value**. Think of it as a labelled box: the label is the name, the size of the box is the data type, and what's inside is the value.

```
// Syntax: dataType variableName = value;
int age = 25;
double price = 9.99;
String name = "Alice";
boolean active = true;
```

1.2 Declaring vs Initializing

You can **declare** a variable first and **initialize** it later, or do both in one statement.

```
// Declaration only
int score;

// Initialization later
score = 100;

// Declaration + initialization in one line
int lives = 3;
```

■ OUTPUT

```
(no output – just storing values in memory)
```

1.3 Variable Naming Rules

- ✔ Must start with a letter, underscore (_), or dollar sign ($)
- ✔ Can contain letters, digits, underscores, dollar signs
- ✔ Case-sensitive: `age` ≠ `Age`
- ✘ Cannot use Java reserved keywords (int, class, if …)

✘ Cannot start with a digit (e.g. `1name` is invalid)

■ Best Practice – camelCase

Java convention uses camelCase for variable names: firstName, totalPrice, isLoggedIn. Constants use ALL_CAPS: MAX_SIZE, PI.

```
// Valid names
int playerScore = 0;
double _taxRate = 0.18;
final int MAX_LIVES = 3;

// Invalid names (compile errors)
// int 1score = 5; ← starts with digit
// int class = 10; ← reserved keyword
```

Chapter 2

Data Types

Defining the kind of data a variable can hold

2.1 Primitive Data Types

Java has 8 built-in primitive types. They store simple values directly in memory (not as objects).

Type	Size	Range / Values	Example Literal
`byte`	8 bit	-128 to 127	`byte b = 10;`
`short`	16 bit	-32,768 to 32,767	`short s = 1000;`
`int`	32 bit	-2,147,483,648 to 2,147,483,647	`int n = 42;`
`long`	64 bit	±9.2 × 10¹■	`long l = 99L;`
`float`	32 bit	~6-7 decimal digits	`float f = 3.14f;`
`double`	64 bit	~15-16 decimal digits	`double d = 3.14159;`
`char`	16 bit	0 to 65,535 (Unicode)	`char c = 'A';`
`boolean`	1 bit	true or false	`boolean x = true;`

```
// Primitive types in action
public class DataTypesDemo {
public static void main(String[] args) {
int age = 20;
double gpa = 3.85;
char grade = 'A';
boolean passed = true;
long bigNum = 9876543210L;

System.out.println("Age: " + age);
System.out.println("GPA: " + gpa);
System.out.println("Grade: " + grade);
System.out.println("Passed: " + passed);
}
```

```
}
```

■ OUTPUT

```
Age: 20
GPA: 3.85
Grade: A
Passed: true
```

2.2 Reference Types (String)

Unlike primitives, reference types store the **address** of an object in memory. **String** is the most commonly used reference type in Java.

```
String firstName = "John";
String lastName = "Doe";
String fullName = firstName + " " + lastName;

System.out.println(fullName);
System.out.println("Length: " + fullName.length());
System.out.println("Upper: " + fullName.toUpperCase());
```

■ OUTPUT

```
John Doe
Length: 8
Upper: JOHN DOE
```

Chapter 3

Operators

Performing operations on variables and values

3.1 Arithmetic Operators

```
int a = 10, b = 3;
System.out.println(a + b); // Addition → 13
System.out.println(a - b); // Subtraction → 7
System.out.println(a * b); // Multiplication → 30
System.out.println(a / b); // Division → 3 (integer!)
System.out.println(a % b); // Modulus → 1
System.out.println(a++); // Post-increment → prints 10, a becomes 11
System.out.println(++b); // Pre-increment → prints 4
```

■ **OUTPUT**

```
13
7
30
3
1
10
4
```

3.2 Relational (Comparison) Operators

Relational operators compare two values and always return a **boolean** (true or false).

```
int x = 5, y = 8;
System.out.println(x == y); // Equal → false
System.out.println(x != y); // Not equal → true
System.out.println(x < y); // Less than → true
System.out.println(x > y); // Greater than → false
System.out.println(x <= y); // Less or equal → true
```

```java
System.out.println(x >= y); // Greater or equal→ false
```

■ OUTPUT

```
false
true
true
false
true
false
```

3.3 Logical Operators

```java
boolean p = true, q = false;
System.out.println(p && q); // AND → false (both must be true)
System.out.println(p || q); // OR → true (at least one true)
System.out.println(!p); // NOT → false (inverts)
```

■ OUTPUT

```
false
true
false
```

3.4 Assignment Operators

```java
int n = 10;
n += 5; // n = n + 5 → 15
n -= 3; // n = n - 3 → 12
n *= 2; // n = n * 2 → 24
n /= 4; // n = n / 4 → 6
n %= 4; // n = n % 4 → 2
System.out.println(n);
```

■ OUTPUT

```
2
```

3.5 Ternary Operator

A compact one-liner **if-else**: `condition ? valueIfTrue : valueIfFalse`

```java
int age = 18;
String status = (age >= 18) ? "Adult" : "Minor";
System.out.println(status);

int a = 7, b = 3;
int max = (a > b) ? a : b;
System.out.println("Max: " + max);
```

■ **OUTPUT**

```
Adult
Max: 7
```

Chapter 4

Input / Output

Communicating with the user through the console

4.1 Output – System.out

Java provides three main print methods. Use `println` for a line with a newline, `print` without a newline, and `printf` for formatted output.

```
System.out.println("Hello, World!"); // new line
System.out.print("No newline here. ");
System.out.print("Same line.");
System.out.println(); // just a newline

int age = 21;
double gpa = 3.756;
System.out.printf("Age: %d, GPA: %.2f%n", age, gpa);
```

■ OUTPUT

```
Hello, World!
No newline here. Same line.

Age: 21, GPA: 3.76
```

printf Format Specifiers

Specifier	Type	Example	Output
%d	Integer	printf("%d", 42)	42
%f	Float / Double	printf("%.2f", 3.14159)	3.14
%s	String	printf("%s", "Hi")	Hi
%c	Char	printf("%c", 'J')	J
%b	Boolean	printf("%b", true)	true
%n	Newline	(platform-independent)	
%5d	Right-align int	printf("%5d", 7)	7

4.2 Input – Scanner Class

The **Scanner** class (from `java.util`) reads user input from the keyboard. Always import it at the top of your file.

```
import java.util.Scanner;

public class InputDemo {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter your name: ");
String name = sc.nextLine();

System.out.print("Enter your age: ");
int age = sc.nextInt();

System.out.printf("Hello %s! You are %d years old.%n", name, age);
sc.close();
}
}
```

■ OUTPUT

```
Enter your name: Alice
Enter your age: 22
Hello Alice! You are 22 years old.
```

Scanner Reading Methods

Method	Reads	Example
`nextLine()`	Full line of text (String)	`sc.nextLine()`
`next()`	Single word (String)	`sc.next()`
`nextInt()`	Integer	`sc.nextInt()`
`nextDouble()`	Double	`sc.nextDouble()`
`nextBoolean()`	Boolean (true/false)	`sc.nextBoolean()`

■ nextLine() after nextInt()

After calling nextInt() or nextDouble(), call sc.nextLine() once to consume the leftover newline character, otherwise the next nextLine() call will return an empty string.

Chapter 5

Type Casting

Converting values from one data type to another

5.1 Implicit Casting (Widening)

Java automatically converts a **smaller** type to a **larger** one. No data is lost — it's safe and happens without any extra code.


```
// Implicit (widening) – no cast needed
int myInt = 100;
long myLong = myInt; // int → long ✔
double myDouble = myLong; // long → double ✔

System.out.println(myInt);
System.out.println(myLong);
System.out.println(myDouble);
```

■ OUTPUT

```
100
100
100.0
```

5.2 Explicit Casting (Narrowing)

When converting from a **larger** type to a **smaller** one, you must write the target type in parentheses. Data **may be lost** — decimals are truncated, or values may overflow.

```
// Explicit (narrowing) – cast required
double pi = 3.14159;
int piInt = (int) pi; // decimal part lost!

long bigValue = 130L;
byte small = (byte) bigValue; // 130 fits in byte (max 127? overflow!)

System.out.println(piInt);
```

```
System.out.println(small); // 130 → -126 (overflow)
```

■ OUTPUT

```
3
-126
```

■ Watch out for overflow!

Casting 130 (a valid int) to byte gives -126 because byte max is 127. Java wraps around: 130 - 256 = -126. Always verify your value fits in the target type before narrowing.

5.3 String ↔ Number Conversion

```
// String to number
String s = "42";
int i = Integer.parseInt(s);
double d = Double.parseDouble("3.14");

// Number to String
String s1 = String.valueOf(100);
String s2 = Integer.toString(200);
String s3 = "" + 300; // quick trick

System.out.println(i + 8); // 50 (numeric add)
System.out.println(s1 + s2); // 100200 (string concat!)
```

■ OUTPUT

```
50
100200
```

■ String + number = concatenation

When you write "" + 300, Java converts the number to a String first. This is why "1" + 2 gives "12", not 3. Use Integer.parseInt() when you need actual arithmetic.

5.4 Quick Casting Reference

From	To	Method	Example	Safe?
`int`	`double`	Implicit	`double d = myInt;`	✔ Yes
`double`	`int`	Explicit (int)	`int i = (int) myDouble;`	■ Truncates

`String`	`int`	Integer.parseInt()	`int i = Integer.parseInt(s);`	■ May throw
`int`	`String`	String.valueOf()	`String s = String.valueOf(n);`	✔ Yes
`char`	`int`	Implicit	`int n = myChar;`	✔ Yes
`int`	`char`	Explicit (char)	`char c = (char) 65;`	✔ If valid

5.5 Putting It All Together

```
import java.util.Scanner;

public class CastingDemo {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter a decimal number: ");

double num = sc.nextDouble();

int truncated = (int) num; // explicit cast

String asString = String.valueOf(num); // to String

System.out.println("Original: " + num);

System.out.println("Truncated: " + truncated);

System.out.println("As String: " + asString);

sc.close();

}

}
```

■ **OUTPUT**

```
Enter a decimal number: 9.75

Original: 9.75

Truncated: 9

As String: 9.75
```